

■ OzZoo Simulation Game

AI Copilot Usage Log

NIT2112 Object-Oriented Programming — Victoria University

This document is a curated log of AI copilot interactions during the design and development of the OzZoo Australian Wildlife Park Simulation. It records key prompts, AI outputs, and critical analysis of each interaction.

The full Technical Design Document is attached to this PDF as an embedded file (ozzoo_design.md). Open the PDF in Adobe Acrobat or a compatible reader and navigate to Attachments to access it.

Entry 1 — Generating the Technical Design Document

Purpose:

Used the AI copilot to produce a comprehensive technical design document before writing any code, establishing the full class hierarchy, interfaces, exception structure, and game loop logic.

Prompt Used

```
I'm a student in NIT2112 Object-Oriented Programming at Victoria University. I need to build 'OzZoo' – an Australian zoo simulation game in Python as my final assignment (worth 50% total: 30% project, 20% presentation). Based on these requirements, generate a technical design document that covers: 1. A full UML class diagram showing all classes and their relationships 2. How each assessed criterion maps to concrete implementation decisions 3. The complete class hierarchy (Animal > Mammal/Bird/Reptile > species) 4. Interface and ABC definitions for ICleanable and IFeedable 5. The custom exception hierarchy (ZooError and subclasses) 6. The Factory pattern implementation for animal creation 7. The daily game loop logic and how events cascade 8. Resource management design (budget, food stock, visitor revenue)
```

AI Output Summary

The AI produced a detailed multi-section technical document including:

- A 3-level class hierarchy for Animal (Mammal, Bird, Reptile) and Enclosure subtypes
- Full ASCII UML diagrams for all major classes and their relationships
- Python code scaffolding for ICleanable, IFeedable, and IObservable interfaces
- A ZooError exception hierarchy with 8 specific exception types
- AnimalFactory implementation using a class-level registry dictionary
- Daily game loop pseudocode covering hunger, health, revenue, and breeding

- Development time estimates per component (~14 hours total)

Analysis

The output was well-structured and directly aligned with the rubric criteria. The AI correctly identified that demonstrating each OOP principle required explicit design decisions — for example, using ABCs rather than plain classes for Animal and Enclosure to satisfy the Abstraction requirement.

Modifications made after reviewing the output:

- Added IObservable interface (Observer pattern) — the AI initially only included Factory
- Adjusted the exception hierarchy to add AnimalSickError, which was missing
- Refined the daily cycle logic to separate morning alerts from evening calculations

Limitation noted: The AI's UML diagrams were ASCII-only. For the submission's Tutorial Document a proper graphical UML diagram will need to be created separately.

Entry 2 — Generating the Student Implementation Guide

Purpose:

After confirming the technical design, used the AI to translate the design document into a beginner-friendly implementation guide with code scaffolding and step-by-step instructions.

Prompt Used

```
I have a completed technical design document for OzZoo (attached). Now generate a student-friendly implementation guide that: - Explains each OOP concept in plain language with the zoo as the analogy - Provides starter code scaffolding for each major class - Gives a step-by-step 'getting started' checklist - Includes example CLI output so I can visualise the end result - Uses an encouraging tone appropriate for a student audience Prioritise demonstrating the OOP concepts clearly over game complexity.
```

AI Output Summary

- Plain-English explanations of inheritance, polymorphism, encapsulation, ABCs
- Class scaffolding for Animal, Enclosure, Zoo, and AnimalFactory
- A 10-step getting started checklist ordered by dependency
- Sample CLI output showing the menu and a feed interaction
- Tips on commenting style (explain 'why' not 'what')

Analysis

The guide was effective as a reference during development. The AI correctly prioritised OOP demonstration over feature richness, which aligned with the rubric weighting (OOP Design = 8/30 project points).

One area where the AI's output needed revision: the scaffolding for the Factory pattern used instance methods rather than class methods. Changed `create_animal()` to a `@classmethod` to match standard Factory pattern conventions and the design document spec.

Entry 3 — Debugging Mission Progress Logic

Purpose:

Used AI to diagnose a bug in the KEEP_HAPPY mission type where the progress bar was not advancing correctly despite animals meeting the happiness threshold.

Prompt Used

My KEEP_HAPPY mission tracks how many consecutive days animals stay happy. The progress bar isn't advancing even when all animals are above 60 happiness. The issue is somewhere in `get_progress_percentage()` or `update_mission_progress()`. Here is a high-level description of the bug: - `get_progress_percentage()` divides `days_elapsed` by `animal_count` (wrong divisor) - `update_mission_progress()` checks completion against `animal_count` not `target_days` Explain the root cause clearly so I can paste this into my coding agent.

AI Output Summary

The AI correctly identified two distinct bugs:

- Bug 1 (`get_progress_percentage`): The divisor was `self._animal_count` instead of `self._target_days`, conflating the metric being tracked (days elapsed) with an unrelated threshold (number of animals).
- Bug 2 (`update_mission_progress`): Completion was checked against `animal_count` rather than `target_days`, meaning a zoo with 5 animals would 'complete' a 'keep happy for 10 days' mission after only 5 days.

Analysis

The AI's diagnosis was accurate and well-explained. The output was directly usable as a problem description for the CLI coding agent, which then implemented the fix correctly. This was the most efficient AI interaction in the project — the AI's strength in explaining logical errors clearly saved approximately 30-45 minutes of manual debugging.

Key learning: Providing a high-level problem description (rather than raw code) produced a more useful and transferable explanation.

Summary of AI Copilot Interactions

Entry	Task	Usefulness	Modifications Required
1	Technical Design Document	High	Added Observer pattern, <code>AnimalSickError</code>
2	Implementation Guide	High	Fixed Factory to use <code>@classmethod</code>
3	Debug KEEP_HAPPY mission	Very High	None — used directly

Overall, AI copilot assistance was most valuable during the design phase and debugging phase. Generated code consistently required review and modification, particularly around design pattern conventions. Critical engagement with AI output — rather than blind acceptance — was essential to producing correct OOP implementations.